

Build Task — Versioned Release Gate for Project 5

— Hints & Solution

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (how a real gate makes itself hard to accidentally bypass). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds. This Build Task pulls together everything from the 5 practice exercises above it — a version log, a gate, a live pointer that resists tampering, and a rollback that goes back to the last version *known to be good* — into one working system.

- [Hint 1 — What you're building, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)
- [Solution](#)

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — What you're building, and the exact pieces

Basic Version

You're building 4 small files that work together: one that saves and loads named versions of a prompt/setting (`versioning.py`), one that's the actual gate — testing a version and deciding whether it gets to go live (`deploy_gate.py`), one that undoes a bad release (`rollback.py`), and one that proves all of it actually works (`test_deploy_gate.py`).

The key idea tying it all together: "what's live right now" should never be a single fact that lives in one plain file someone could edit by hand. It should be the *answer to a question* — "looking at everything that's ever happened, what's the most recent version that actually passed?" — worked out from a running history, the same way `gate_bypass` and `rollback_trigger` already got you thinking.

Here are the exact pieces you need to look up and use:

- A `@dataclass` (from `dataclasses`) for `VersionRecord` and `DeployResult` — clean, typed containers for the data moving between these files.
- `json.dump()` / `json.load()` — same as every earlier exercise, for saving the version log and the history of deploy/rollback events.
- `datetime.now(timezone.utc).isoformat()` — for timestamps; ISO-format strings also happen to sort correctly with plain `<` and `>`, which matters for "what was live at time T."
- Your `run_eval_suite()` stand-in from earlier exercises, now taking a whole `config` dict instead of just a prompt string — real Project 5 settings are more than one string.
- `pytest` — a test function is just `def test_something(): assert some_condition;` `pytest` finds and runs every function starting with `test_` in a file.

Intermediate Version

Think of this Build Task as 3 real jobs, not 1: **storing** versions (`versioning.py`), **gating** a release (`deploy_gate.py`), and **recovering** from a bad one (`rollback.py`) — each file does one job, and each reads/writes the same underlying data, not 3 separate copies of the truth.

The design that makes "the gate can't be skipped by accident" actually true: instead of a `live_version.txt` that any text editor can rewrite with zero friction, keep an **append-only event log** — every `deploy()` call, pass or fail, appends one event; every `rollback_to_previous()` call appends one more. "What's live right now" is never stored directly — it's *derived*, every time, as "the version named in the most recent `deploy_pass` or `rollback` event." There's no separate pointer file left for someone to edit without realizing there's a gate behind it at all.

The exact pieces:

- `save_version(name: str, config: dict) -> VersionRecord` and `get_version(name: str) -> VersionRecord | None` in `versioning.py` — a thin, reusable layer other files build on, same shape as the `version_log` exercise.
- `deploy(version_name: str, threshold: float) -> DeployResult` in `deploy_gate.py` — looks up the version, runs `run_eval_suite(config)`, and appends a `deploy_pass` or `deploy_fail` event to the history depending on the score, exactly like `release_gate`'s exercise, but recording an event instead of writing a pointer file.
- `get_live_version() -> str | None` and `get_live_version_at(timestamp: str) -> str | None` — both scan the same event history; the second just ignores every event after `timestamp`.
- `rollback_to_previous() -> VersionRecord` in `rollback.py` — finds the currently live version from the history, then walks backward through earlier `deploy_pass` / `rollback` events to find the most recent *different* version, and appends a `rollback` event pointing at it. This is exactly `rollback_trigger`'s idea of "go back to the last version *known to have passed*," now wired to real version records instead of a stub.
- `pytest` basics: a test file imports the functions under test, and each `test_...` function sets up a small scenario and asserts the result — `tmp_path`, a built-in `pytest` fixture, gives each test its own throwaway directory so tests don't interfere with each other's `versions.json` / `history.json` files.

Advanced Version

Two real design questions separate a Build Task that technically works from one you'd actually trust in front of real users.

First — does the event log itself resist tampering, or did you just move the same problem one file over? An append-only history is already much better than a single mutable pointer (there's no side door that looks unrelated to the gate), but nothing yet stops someone from opening `history.json` and appending a fake `deploy_pass` event by hand — same failure mode `gate_bypass` warned about, just against a different file. The fix is the same one that exercise landed on: sign every event with `hmac`, using a secret that isn't stored inside `history.json` itself, and have every reader verify the signature before trusting an event. A hand-appended event without a valid signature gets treated as untrusted, not silently believed.

Second — what does `deploy()` actually depend on, and can `test_deploy_gate.py` test it without needing your real, slow, possibly-costly Doc13 suite to run on every test? Real tests should be fast and not depend on an outside API. The standard fix is **dependency injection**: instead of `deploy()` calling a hardcoded `run_eval_suite`, let it accept the eval function as a parameter (defaulting to the real one), so tests can pass in a small, fast, fake one instead — this is also just `pytest`'s `monkeypatch` fixture applied to a function, if you'd rather patch it than pass it as a parameter.

The extra pieces needed:

- `hmac.new(secret.encode(), message.encode(), hashlib.sha256).hexdigest()` — sign each event's `f"{event_type}:{version_name}:{at}"` string, store the signature alongside the event, and verify it wherever the history is read.
- `def deploy(version_name, threshold, eval_fn=run_eval_suite) -> DeployResult:` — a default parameter that real callers never need to touch, but tests can override freely.
- `pytest.raises(SomeError)` — the standard way to assert a function raises the exception you expect, used for `test_deploy_gate.py`'s "no such version" and "nothing to roll back to" cases.

Sketch the signed-event append and a `deploy()` with an injectable `eval_fn` yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic names the 4 files, their jobs, and the exact tools — dataclasses, JSON, timestamps, the eval stub, `pytest` — in plain words. Intermediate explains the append-only event-log design that answers "what's live" by derivation instead of storing it directly, and how `rollback_to_previous()` walks that history to find the last version that actually passed. Advanced asks the two harder questions underneath both files — can the event log itself be faked, and can this be tested without depending on your real, slow test suite — and answers both: `hmac`-signed events, and an injectable `eval_fn` parameter.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
versioning.py:
    VersionRecord holds: version_name, config, created_at

    function save_version(name, config):
        load existing versions list (or start empty)
        append a new VersionRecord
        save the whole list back
        return the new record

    function get_version(name):
        search the loaded list for a matching version_name
        return it, or None if not found

deploy_gate.py:
    DeployResult holds: version_name, passed, score, failing_cases

    function deploy(version_name, threshold):
        look up the version with get_version()
        if not found: raise an error
        run the eval suite against its config
        passed = score >= threshold
        record a "deploy_pass" or "deploy_fail" event in history.json, with a timestamp
        return a DeployResult

    function get_live_version():
        read history.json
        find the most recent event that is "deploy_pass" or "rollback"
        return its version_name (or None if there isn't one)

rollback.py:
    function rollback_to_previous():
        find the current live version from history
        walk backward through history for an earlier deploy_pass/rollback
        event with a DIFFERENT version_name
        if found: record a "rollback" event pointing at it, return that version
        if not found: raise an error
```

Here's almost the whole thing for `deploy_gate.py` — just try running it and reading it line by line:

```
import json
from datetime import datetime, timezone

def run_eval_suite(config):
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def record_event(event_type, version_name, path="history.json"):
    try:
        with open(path) as f:
            history = json.load(f)
    except FileNotFoundError:
        history = []
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def deploy(version_name, threshold):
    from versioning import get_version
    record = get_version(version_name)
    result = run_eval_suite(record["config"])
    passed = result["score"] >= threshold
    if passed:
        record_event("deploy_pass", version_name)
    else:
        record_event("deploy_fail", version_name)
    return {"version_name": version_name, "passed": passed, "score": result["score"],
            "failing_cases": result["failed"]}
```

Expected output, given a saved version `"v1"` with `config={"prompt": "Answer fast."}`, calling `deploy("v1", 0.9)`: `{'version_name': 'v1', 'passed': False, 'score': 0.6, 'failing_cases': ['greeting_case', 'citation_case']}`.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
versioning.py:
    @dataclass VersionRecord: version_name: str, config: dict, created_at: str

    function load_versions(path) -> list[VersionRecord]
    function save_version(name, config, path) -> VersionRecord
    function get_version(name, path) -> VersionRecord | None

deploy_gate.py:
    @dataclass DeployResult: version_name: str, passed: bool, score: float, faili

    class VersionNotFoundError(Exception)

    function load_history(path) -> list[dict]
    function record_event(event_type, version_name, path) -> None
    function get_live_version(path) -> str | None
    function get_live_version_at(timestamp, path) -> str | None
    function deploy(version_name, threshold, versions_path, history_path) -> Depl

rollback.py:
    class NoPreviousVersionError(Exception)
    function rollback_to_previous(history_path, versions_path) -> VersionRecord
```

```
# versioning.py
import json
from dataclasses import dataclass, asdict
from datetime import datetime, timezone
from pathlib import Path

@dataclass
class VersionRecord:
    version_name: str
    config: dict
    created_at: str

def load_versions(path: str = "versions.json") -> list[VersionRecord]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw = json.load(f)
```

```

    records = []
    for item in raw:
        records.append(VersionRecord(**item))
    return records

def save_version(name: str, config: dict, path: str = "versions.json") -> VersionRecord:
    versions = load_versions(path)
    record = VersionRecord(
        version_name=name,
        config=config,
        created_at=datetime.now(timezone.utc).isoformat(),
    )
    versions.append(record)
    with open(path, "w") as f:
        json.dump([asdict(v) for v in versions], f, indent=2)
    return record

def get_version(name: str, path: str = "versions.json") -> VersionRecord | None:
    for v in load_versions(path):
        if v.version_name == name:
            return v
    return None

```

```

# deploy_gate.py
import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from versioning import get_version

class VersionNotFoundError(Exception):
    pass

@dataclass
class DeployResult:
    version_name: str
    passed: bool
    score: float
    failing_cases: list

def run_eval_suite(config: dict) -> dict:
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_history(path: str = "history.json") -> list[dict]:

```

```

    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def record_event(event_type: str, version_name: str, path: str = "history.json")
    history = load_history(path)
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def get_live_version(path: str = "history.json") -> str | None:
    return get_live_version_at(datetime.now(timezone.utc).isoformat(), path)

def get_live_version_at(timestamp: str, path: str = "history.json") -> str | None
    passing = []
    for event in load_history(path):
        if event["event"] in ("deploy_pass", "rollback") and event["at"] <= times
            passing.append(event)
    if not passing:
        return None
    return passing[-1]["version_name"]

def deploy(version_name: str, threshold: float, versions_path: str = "versions.js
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(record.config)
    passed = result["score"] >= threshold

    if passed:
        record_event("deploy_pass", version_name, history_path)
    else:
        record_event("deploy_fail", version_name, history_path)

    return DeployResult(
        version_name=version_name,
        passed=passed,
        score=result["score"],
        failing_cases=result["failed"],
    )

```

Now write `rollback.py`'s `rollback_to_previous()` yourself, using the same walk-backward-through-history idea from Hint 1, and compare against the [Solution](#).

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

deploy_gate.py additions:

```
SECRET = a value not stored inside history.json
```

```
function sign_event(event_type, version_name, at) -> str:
    message = f"{event_type}:{version_name}:{at}"
    return hmac.new(SECRET, message, sha256).hexdigest()
```

```
record_event now also computes and stores "signature": sign_event(...)
```

```
function load_history(path):
    for every event, recompute the expected signature
    if it doesn't match what's stored: drop the event (or flag it), don't trust it
    return only the trustworthy events
```

```
deploy() gains an eval_fn parameter, defaulting to the real run_eval_suite,
so test_deploy_gate.py can pass in a fast fake instead
```

Here's the signing piece, almost complete — fill in `load_history`'s verification step yourself:

```
import hmac
import hashlib

SECRET = "replace-with-a-real-secret-not-committed-to-git"

def sign_event(event_type: str, version_name: str, at: str) -> str:
    message = f"{event_type}:{version_name}:{at}"
    return hmac.new(SECRET.encode(), message.encode(), hashlib.sha256).hexdigest()

def record_event(event_type: str, version_name: str, path: str = "history.json"):
    history = load_history(path)
    at = datetime.now(timezone.utc).isoformat()
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": at,
        "signature": sign_event(event_type, version_name, at),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)
```

```
def load_history(path: str = "history.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw_events = json.load(f)

    # your turn: build a new list containing only the events whose stored
    # "signature" matches sign_event(event["event"], event["version_name"],
    # event["at"]) via hmac.compare_digest — skip (don't include) any event
    # that fails this check
    ...
```

And here's `deploy()` with an injectable `eval_fn`:

```
def deploy(
    version_name: str,
    threshold: float,
    eval_fn=run_eval_suite,
    versions_path: str = "versions.json",
    history_path: str = "history.json",
) -> DeployResult:
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = eval_fn(record.config)
    passed = result["score"] >= threshold
    event_type = "deploy_pass" if passed else "deploy_fail"
    record_event(event_type, version_name, history_path)

    return DeployResult(version_name, passed, result["score"], result["failed"])
```

Fill in `load_history`'s signature check yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic's pseudocode and near-complete code get the append-only event idea working for the tidy case, with no thought for how `deploy_gate.py` and `versioning.py` should actually split up, or how a test would call any of it. Intermediate turns that into a real, typed, multi-file design — `VersionRecord`, `DeployResult`, proper history helpers — matching the Build Task's suggested file layout exactly. Advanced adds what the Build Task's own constraint demands but Intermediate doesn't yet satisfy on its own — signed events so a hand-appended fake event is detectable, not just structurally-plausible — plus an injectable `eval_fn`, which is what makes `test_deploy_gate.py` fast and independent of your real Doc13 suite.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Solution

Every code block below shows real, runnable behavior. Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to build the same gate, with real tradeoffs between them.

Basic Version

Approach 1 — one file, the direct way

```
# deploy_gate.py – everything in one file for this first pass
import json
from datetime import datetime, timezone
from pathlib import Path

def run_eval_suite(config):
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_versions(path="versions.json"):
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def save_version(name, config, path="versions.json"):
    versions = load_versions(path)
    versions.append({"version_name": name, "config": config, "created_at": str(datetime.now())})
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)

def get_version(name, path="versions.json"):
    for v in load_versions(path):
        if v["version_name"] == name:
            return v
    return None

def load_history(path="history.json"):
    if not Path(path).exists():
        return []
```

```

with open(path) as f:
    return json.load(f)

def record_event(event_type, version_name, path="history.json"):
    history = load_history(path)
    history.append({"event": event_type, "version_name": version_name, "at": date
with open(path, "w") as f:
    json.dump(history, f, indent=2)

def get_live_version(path="history.json"):
    passing = []
    for e in load_history(path):
        if e["event"] in ("deploy_pass", "rollback"):
            passing.append(e)
    if not passing:
        return None
    return passing[-1]["version_name"]

def deploy(version_name, threshold):
    record = get_version(version_name)
    if record is None:
        print("No such version:", version_name)
        return None

    result = run_eval_suite(record["config"])
    if result["score"] >= threshold:
        record_event("deploy_pass", version_name)
        print(version_name, "deployed, score", result["score"])
    else:
        record_event("deploy_fail", version_name)
        print(version_name, "NOT deployed, score", result["score"], "failing:", r
    return result

def rollback_to_previous():
    history = load_history()
    passing = []
    for e in history:
        if e["event"] in ("deploy_pass", "rollback"):
            passing.append(e)
    if not passing:
        print("Nothing has ever passed the gate.")
        return None

    current_live = passing[-1]["version_name"]
    for e in reversed(passing[:-1]):
        if e["version_name"] != current_live:
            record_event("rollback", e["version_name"])

```

```

        print("Rolled back to", e["version_name"])
        return e["version_name"]

    print("No earlier passing version to roll back to.")
    return None

save_version("v1", {"prompt": "Answer the customer's question."})
save_version("v2", {"prompt": "Answer the customer's question politely and cite t

deploy("v1", 0.5)
deploy("v2", 0.9)
print("live:", get_live_version())
rollback_to_previous()
print("live:", get_live_version())

```

Expected output:

```

v1 deployed, score 0.6
v2 deployed, score 1.0
live: v2
Rolled back to v1
live: v1

```

This version works correctly for every requirement and constraint in the Build Task — it's one big file instead of the suggested 4, uses plain dicts instead of dataclasses, `rollback_to_previous()` returns just the version name instead of a full `VersionRecord`, and there's no `test_deploy_gate.py` yet. All fine for a first working version; Intermediate splits it apart properly and fixes the return type.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — split into `versioning.py` / `deploy_gate.py` / `rollback.py` , with dataclasses

`versioning.py`

```
import json
from dataclasses import dataclass, asdict
from datetime import datetime, timezone
from pathlib import Path

@dataclass
class VersionRecord:
    version_name: str
    config: dict
    created_at: str

def load_versions(path: str = "versions.json") -> list[VersionRecord]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw = json.load(f)
    records = []
    for item in raw:
        records.append(VersionRecord(**item))
    return records

def save_version(name: str, config: dict, path: str = "versions.json") -> VersionRecord:
    versions = load_versions(path)
    record = VersionRecord(version_name=name, config=config, created_at=datetime.now(timezone.utc).isoformat())
    versions.append(record)
    with open(path, "w") as f:
        json.dump([asdict(v) for v in versions], f, indent=2)
    return record

def get_version(name: str, path: str = "versions.json") -> VersionRecord | None:
    for v in load_versions(path):
        if v.version_name == name:
            return v
    return None
```

deploy_gate.py

```

import json
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
from versioning import get_version

class VersionNotFoundError(Exception):
    pass

@dataclass
class DeployResult:
    version_name: str
    passed: bool
    score: float
    failing_cases: list

def run_eval_suite(config: dict) -> dict:
    if "politely" in config.get("prompt", ""):
        return {"score": 1.0, "failed": []}
    return {"score": 0.6, "failed": ["greeting_case", "citation_case"]}

def load_history(path: str = "history.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        return json.load(f)

def record_event(event_type: str, version_name: str, path: str = "history.json"):
    history = load_history(path)
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def get_live_version(path: str = "history.json") -> str | None:
    return get_live_version_at(datetime.now(timezone.utc).isoformat(), path)

def get_live_version_at(timestamp: str, path: str = "history.json") -> str | None:
    passing = []
    for event in load_history(path):
        if event["event"] in ("deploy_pass", "rollback") and event["at"] <= times
        passing.append(event)

```

```

    if not passing:
        return None
    return passing[-1]["version_name"]

def deploy(version_name: str, threshold: float, versions_path: str = "versions.js")
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(record.config)
    passed = result["score"] >= threshold
    event_type = "deploy_pass" if passed else "deploy_fail"
    record_event(event_type, version_name, history_path)

    return DeployResult(version_name=version_name, passed=passed, score=result["s

```

rollback.py

```

from versioning import get_version
from deploy_gate import load_history, record_event

class NoPreviousVersionError(Exception):
    pass

def rollback_to_previous(history_path: str = "history.json", versions_path: str = "versions.js")
    history = load_history(history_path)
    passing = []
    for event in history:
        if event["event"] in ("deploy_pass", "rollback"):
            passing.append(event)

    if not passing:
        raise NoPreviousVersionError("No version has ever passed the gate – nothing to rollback")

    current_live = passing[-1]["version_name"]
    previous = None
    for event in reversed(passing[:-1]):
        if event["version_name"] != current_live:
            previous = event["version_name"]
            break

    if previous is None:
        raise NoPreviousVersionError(f"No earlier passing version before {current_live}")

    record_event("rollback", previous, history_path)
    return get_version(previous, versions_path)

```

`rollback_to_previous()` returns a full `VersionRecord`, not just a bare name — matching the Build Task's stated interface (`rollback.py` → `rollback_to_previous()` → `VersionRecord`), and giving the caller everything about the restored version (its `config`, `created_at`) in one call, not just its name.

```
# main.py – proving it works
from versioning import save_version
from deploy_gate import deploy, get_live_version
from rollback import rollback_to_previous

save_version("v1", {"prompt": "Answer the customer's question."})
save_version("v2", {"prompt": "Answer the customer's question politely and cite t

deploy("v1", 0.5)
deploy("v2", 0.9)
print("live:", get_live_version())

restored = rollback_to_previous()
print("rolled back to:", restored.version_name)
print("live:", get_live_version())
```

Expected output:

```
live: v2
rolled back to: v1
live: v1
```

Approach 2 — `test_deploy_gate.py`, matching the Test Cases table

```
# test_deploy_gate.py
import pytest
from versioning import save_version
from deploy_gate import deploy, get_live_version, VersionNotFoundError
from rollback import rollback_to_previous, NoPreviousVersionError

def test_version_above_bar_releases(tmp_path):
    versions_path = str(tmp_path / "versions.json")
    history_path = str(tmp_path / "history.json")
    save_version("v1", {"prompt": "Answer politely."}, path=versions_path)

    result = deploy("v1", 0.9, versions_path=versions_path, history_path=history_

    assert result.passed is True
    assert get_live_version(history_path) == "v1"
```

```
def test_version_below_bar_refused(tmp_path):
    versions_path = str(tmp_path / "versions.json")
    history_path = str(tmp_path / "history.json")
    save_version("v1", {"prompt": "Answer fast."}, path=versions_path)

    result = deploy("v1", 0.9, versions_path=versions_path, history_path=history_path)

    assert result.passed is False
    assert len(result.failing_cases) > 0
    assert get_live_version(history_path) is None

def test_deploy_unknown_version_raises(tmp_path):
    versions_path = str(tmp_path / "versions.json")
    with pytest.raises(VersionNotFoundError):
        deploy("does-not-exist", 0.9, versions_path=versions_path)

def test_rollback_with_nothing_passed_raises(tmp_path):
    history_path = str(tmp_path / "history.json")
    with pytest.raises(NoPreviousVersionError):
        rollback_to_previous(history_path=history_path)
```

Expected output, running `pytest test_deploy_gate.py -v`:

```
test_deploy_gate.py::test_version_above_bar_releases PASSED
test_deploy_gate.py::test_version_below_bar_refused PASSED
test_deploy_gate.py::test_deploy_unknown_version_raises PASSED
test_deploy_gate.py::test_rollback_with_nothing_passed_raises PASSED
```

`tmp_path` is a built-in `pytest` fixture — a fresh, empty temporary directory handed to any test function that asks for it by name as a parameter, so each test gets its own `versions.json` / `history.json` and can't interfere with another test or with your real project files.

Difference from Basic: Approach 1 splits Basic's single file into the Build Task's suggested 3 modules, each importing only what it needs from the others, and swaps plain dicts for typed `VersionRecord` / `DeployResult` dataclasses — the same upgrade `release_gate`'s Advanced level made. Approach 2 adds the missing 4th file, `test_deploy_gate.py`, with one test per row of the Build Task's own Test Cases table, using `tmp_path` so tests run in complete isolation from real data.

[Back to the Build Task](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — `hmac`-signed history events

```
# deploy_gate.py – signing additions on top of the Intermediate version
import hmac
import hashlib

SECRET = "replace-with-a-real-secret-not-committed-to-git"

def sign_event(event_type: str, version_name: str, at: str) -> str:
    message = f"{event_type}:{version_name}:{at}"
    return hmac.new(SECRET.encode(), message.encode(), hashlib.sha256).hexdigest()

def record_event(event_type: str, version_name: str, path: str = "history.json")
    history = load_history(path)
    at = datetime.now(timezone.utc).isoformat()
    history.append({
        "event": event_type,
        "version_name": version_name,
        "at": at,
        "signature": sign_event(event_type, version_name, at),
    })
    with open(path, "w") as f:
        json.dump(history, f, indent=2)

def load_history(path: str = "history.json") -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
        raw_events = json.load(f)

    trustworthy = []
    for event in raw_events:
        expected = sign_event(event["event"], event["version_name"], event["at"])
        if hmac.compare_digest(event.get("signature", ""), expected):
            trustworthy.append(event)
        else:
            print(f"Warning: dropping untrusted history event: {event}")
    return trustworthy
```

```
# proving a hand-appended fake event gets ignored
save_version("v1", {"prompt": "Answer politely."})
deploy("v1", 0.5)
print("live:", get_live_version())

# an attacker appends a fake event by hand, with no valid signature
import json
with open("history.json") as f:
    history = json.load(f)
history.append({"event": "deploy_pass", "version_name": "totally-fake-version", "
with open("history.json", "w") as f:
    json.dump(history, f, indent=2)

print("live after fake event appended:", get_live_version())
```

Expected output:

```
live: v1
Warning: dropping untrusted history event: {'event': 'deploy_pass', 'version_name
live after fake event appended: v1
```

The fake event is loaded, checked, and dropped — `get_live_version()` still returns `v1`, not the fabricated version, because `load_history()` never lets an unsigned or mis-signed event count as real.

Approach 2 — an injectable `eval_fn`, for fast, isolated tests

```
def deploy(
    version_name: str,
    threshold: float,
    eval_fn=run_eval_suite,
    versions_path: str = "versions.json",
    history_path: str = "history.json",
) -> DeployResult:
    record = get_version(version_name, versions_path)
    if record is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = eval_fn(record.config)
    passed = result["score"] >= threshold
    event_type = "deploy_pass" if passed else "deploy_fail"
    record_event(event_type, version_name, history_path)

    return DeployResult(version_name, passed, result["score"], result["failed"])
```

```
# test_deploy_gate.py, using a fake eval_fn instead of the real (slow, costly) suite
def fake_eval_suite(config):
    return {"score": 1.0, "failed": []}

def test_deploy_with_fake_eval_fn(tmp_path):
    versions_path = str(tmp_path / "versions.json")
    history_path = str(tmp_path / "history.json")
    save_version("v1", {"prompt": "anything at all"}, path=versions_path)

    result = deploy("v1", 0.9, eval_fn=fake_eval_suite, versions_path=versions_path,
                    history_path=history_path)

    assert result.passed is True
    assert result.score == 1.0
```

Expected output, running just this test: `PASSED` — and it never once calls the real `run_eval_suite`, so it stays fast and deterministic no matter how slow or flaky the real Doc13 suite is.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's append-only event history already removes the single easiest bypass (no separate `live_version.txt` to edit without noticing a gate exists), but a hand-appended event with a plausible shape would still be silently trusted. Approach 1 closes that gap with signed events, directly reusing `gate_bypass`'s `hmac` technique. Approach 2 solves a completely different problem — testability — by letting `deploy()`'s eval function be swapped out, which is what makes `test_deploy_gate.py`'s tests fast and independent of your real, possibly slow or costly, Doc13 suite.

Which one should you actually build? For getting Project 5's real gate working, Intermediate Approach 1 plus Approach 2's test file is the right target — it satisfies every Build Task requirement and constraint, and it's what you should actually wire into Project 5's runtime. Add Advanced Approach 1's signed events once you're deploying this where more than one person has write access to the history file — the whole point of `gate_bypass`'s exercise was that "nobody would do that" isn't a real defense. Add Advanced Approach 2's injectable `eval_fn` as soon as your real Doc13 suite is slow enough, or costs enough in API calls, that running it on every single test run stops being something you actually do.